

Report on The Cost of Living (US)

By Devin Frost

In recent years, the US economy has been hit hard by multiple economic crises, from global pandemics to supply chain shocks to wars, and naturally, the expenditure of resources to combat these crises has negatively impacted the cost of living in the US. This report takes a holistic look into the cost of living in the United States, using data from the Economic Policy Institute (EPI). In order to understand the state of the US economy, we first need to measure the cost of living, then normalize it in accordance to median incomes in order to gauge regional affordability.

To prepare my data for this analysis, I dropped rows with null values to simplify the analysis process. It only meant losing 10 records, which should not meaningfully offset the value of this analysis. I also synthesized numerous measures using the dataset. The dataset gives a simple 'family_member_count' field, which isn't provided as a numerical value. I parse the values into the 'parents' and 'children' fields. Further, I sum these up into a 'family_size' value. In order to measure the affordability of a region, I take total expenditures and divide it by median income, to normalize the values.

The family unit is not complete without children, and in the modern, fast-paced world, childcare is a necessity. Figure 1 is a map of childcare costs by state. Childcare costs have considerable variation across the states. Figure 2 is a bar chart of the top 12 most expensive counties in which to raise a child. It includes counties from New York and the District of Columbia.

Housing. We all need it, whether we are high up in the mountains of Appalachia or in the Great Plains - to simply survive the elements. Figure 4 is a histogram that shows the typical annual cost of housing per state. Figure 5 shows housing prices by state on a map, and as you can see, housing prices are generally highest near the coasts, and lowest in the Midwest and Southeast. Hawaii, being a small slice of Heaven on Earth, naturally has the highest housing prices, with a median of 61k / year. Country-wide it floats at a median of \$10,400 each year. Figure 6 is a scatterplot of housing prices versus taxes, and suggests that higher housing costs are positively correlated with higher taxes, which is likely due to property tax law.

A well-functioning economy depends on cheap transportation to move goods, services, and people to where the demand is. Figure 7 shows the distribution curve of transportation costs across America. As you can see, the graph is left skewed with a median of approximately \$13,800 a year, which, as shown by Figure 8 is due to the outlier Washington, D.C., where transportation is cheap. Figure 9 is a regression plot of transportation cost versus total expenditures, and it shows that transportation costs are negatively correlated with total costs.

This may indicate that cheaper transportation leads to more economic growth and therefore higher paying jobs and cost of living

Our quality of life is impacted largely by our health, and consequently, regions with high healthcare costs are indicative of regions that struggle to meet a standard of living. Figure 10 is a histogram of the distribution of healthcare expenditures accross the country. It is somewhat right-skewed, but is largely normally distributed. Figure 11 shows that West Virginia is by far the most expensive place to pay for healthcare, with a median price of \$37,000 annually. It is reflective of poor policy decisions in the region.

Unfortunately, in order to organize society, any state needs a government, and any government needs tax dollars to power programs that power public infrastructure and social programs. Figure 12 is a bar chart of taxes by state, and Figure 13 is a map thereof. As shown by the map, Oregon and Washington, DC have some of the highest taxes in the country. Figure 14 is a bar chart of the top 12 highest-tax counties in the country. Counties from New York and Washington, D.C have the highest taxes of any region in the United States

In order to pay for all of this, people naturally need sturdy, competitive incomes. Figure 15 is a boxplot of median household incomes across the counties of the 50 United States, plus the District of Columbia, and it shows that incomes across the state are highly variable, ranging from \$27,000 in Georgia, up to \$177k in Virginia. Virginia was by far the most riddled with inter-county income inequality. Deleware had the least variation. Figure 16 better conveys this information geographically. Figure 17 is a histogram of median family incomes by state and it is evident that the median wage in the US is approximately \$70,000 a year.

Figure 19 is a correlation matrix between each of the aforementioned factors and concludes that family size is highly correlated with the cost of food and healthcare, that total costs are highly correlated with the cost of food and the cost of extraneus necessities, and, interestingly, housing costs are correlated with extraneuss necessity expenditures. Figure 18 shows that total expenditures are positively correlated with higher taxes. A small table (Figure 20) is provided to illustrate the data that were provided by the Economic Policy Institute (EPI), including the aforementioned feilds `parents`, `children`, `family\_size`, and `is\_foodcost\_outlier`.

Childcare

Figure 1



Figure 2

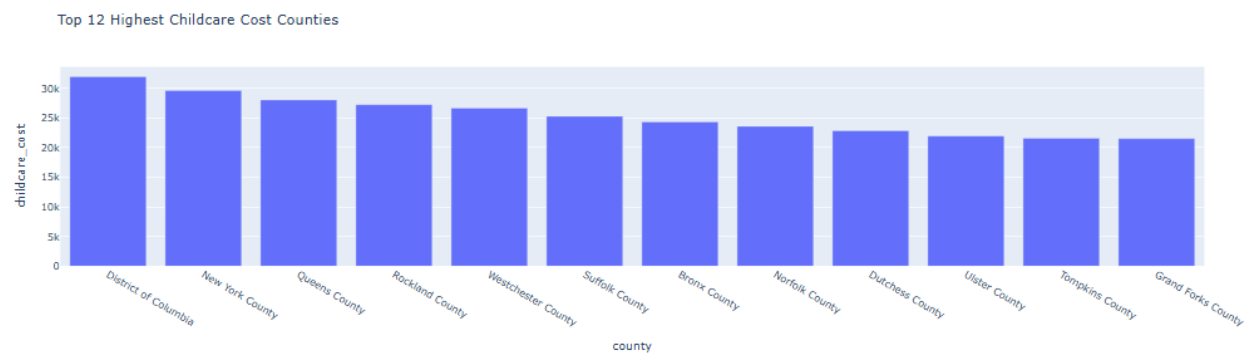
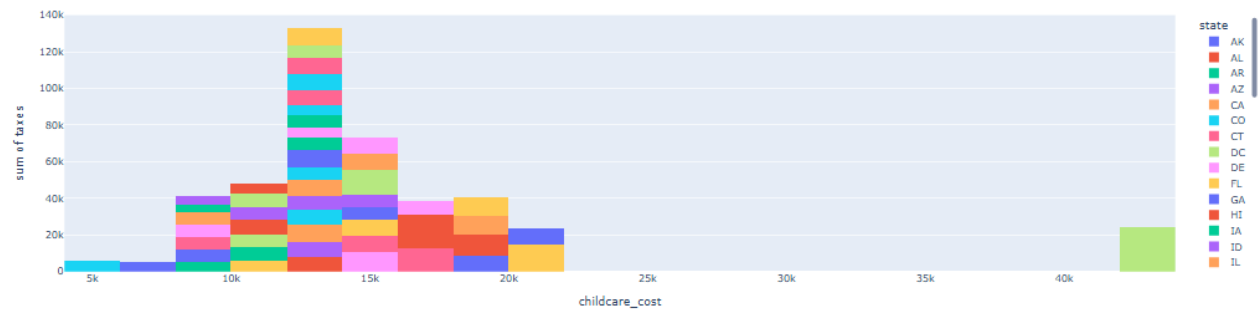


Figure 3



Housing

Figure 4

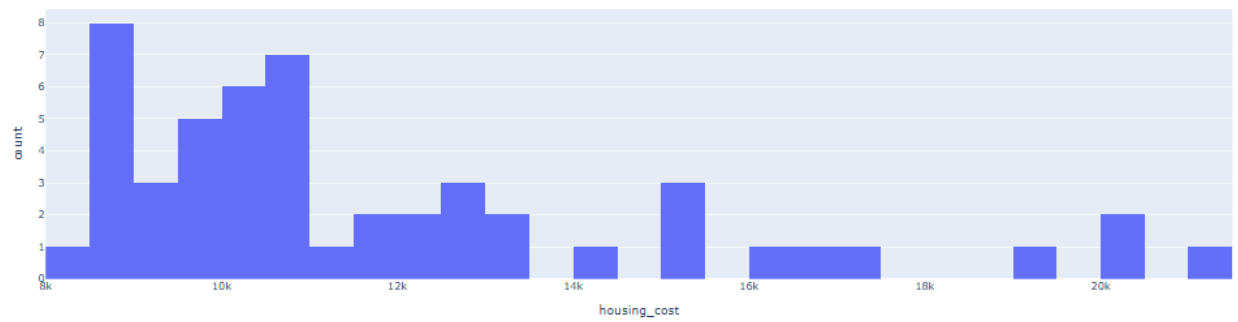


Figure 5

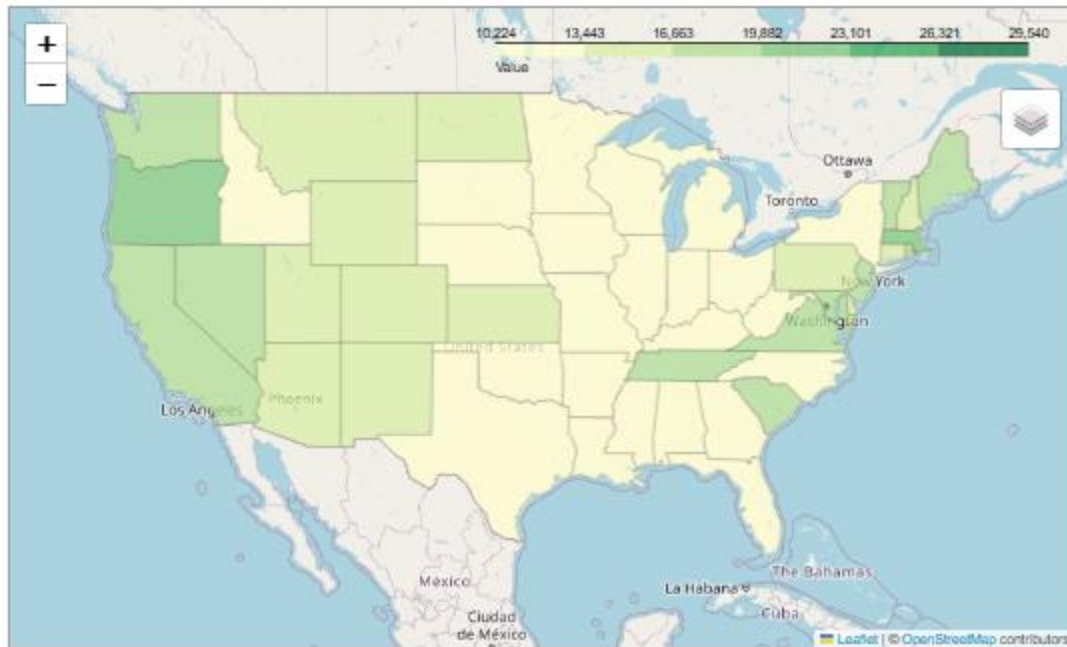
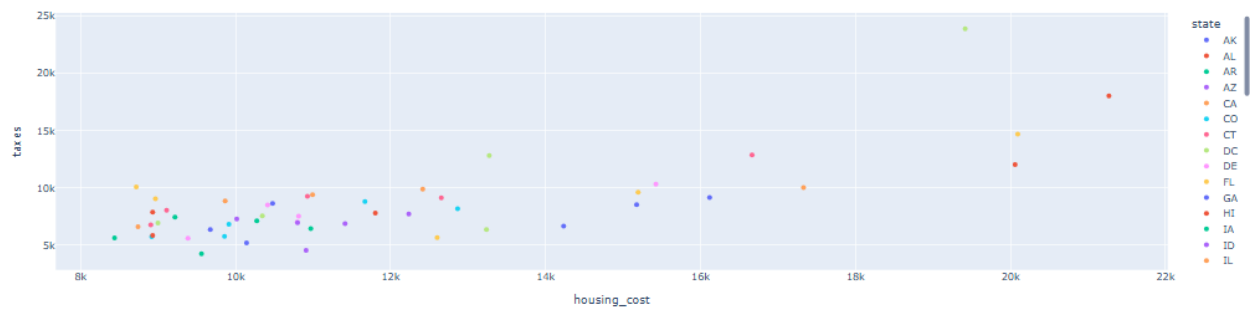


Figure 6



Transportation

Figure 7

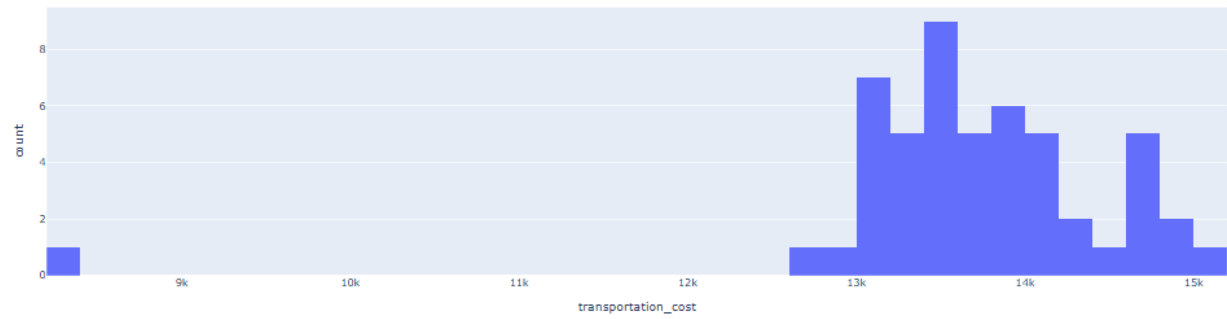


Figure 8

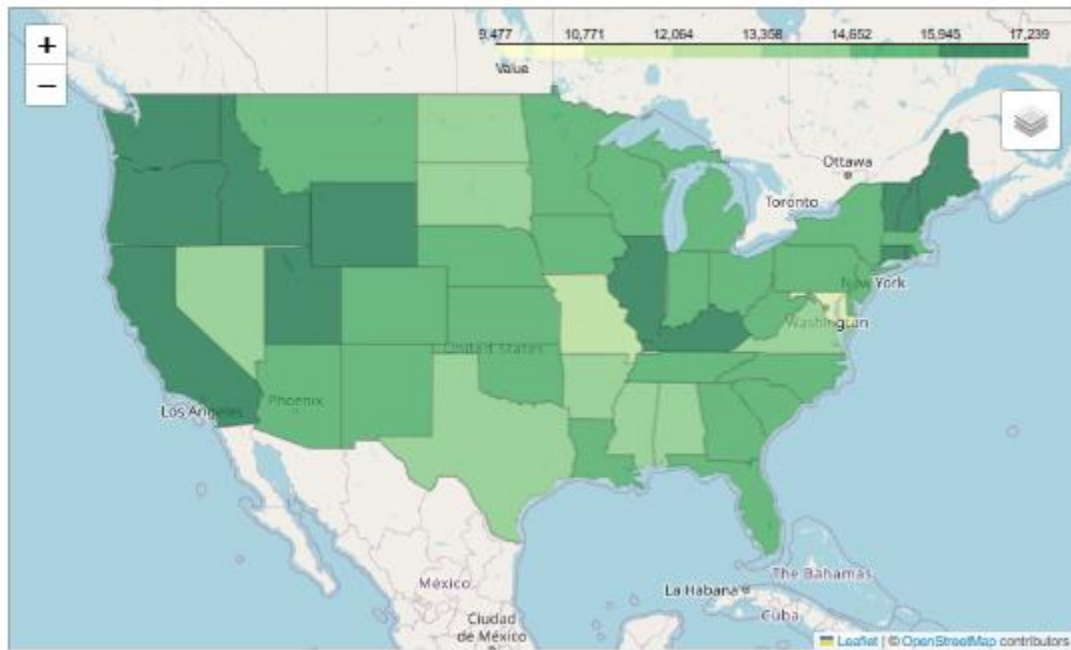
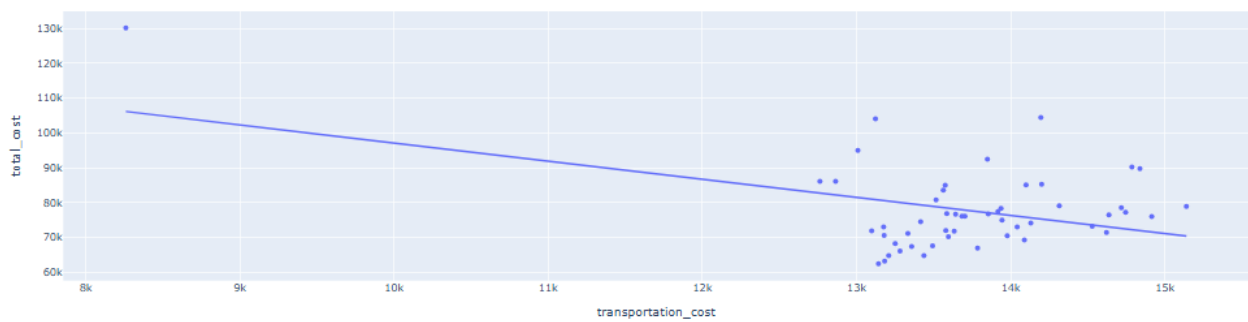


Figure 9



Healthcare

Figure 10

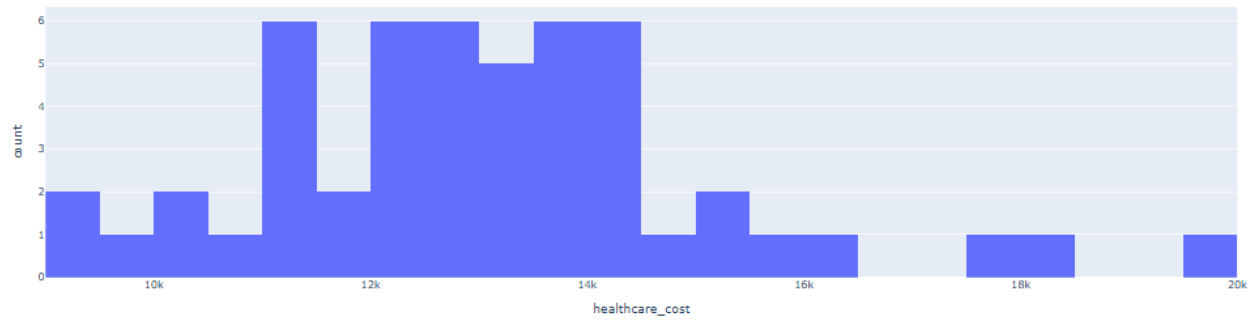
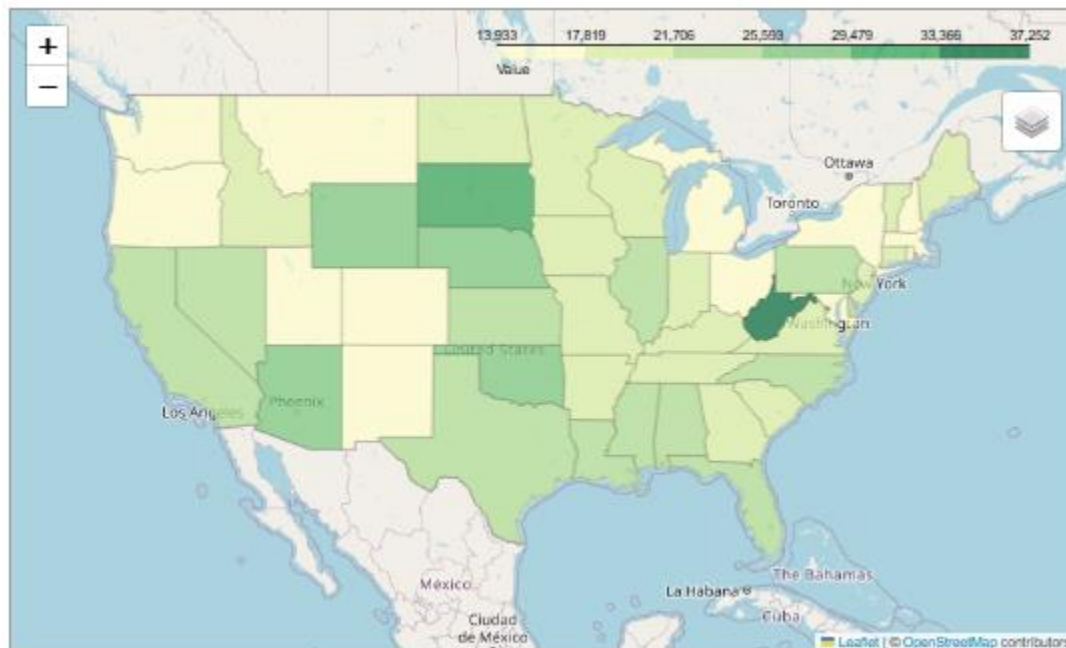


Figure 11



Taxes

Figure 12

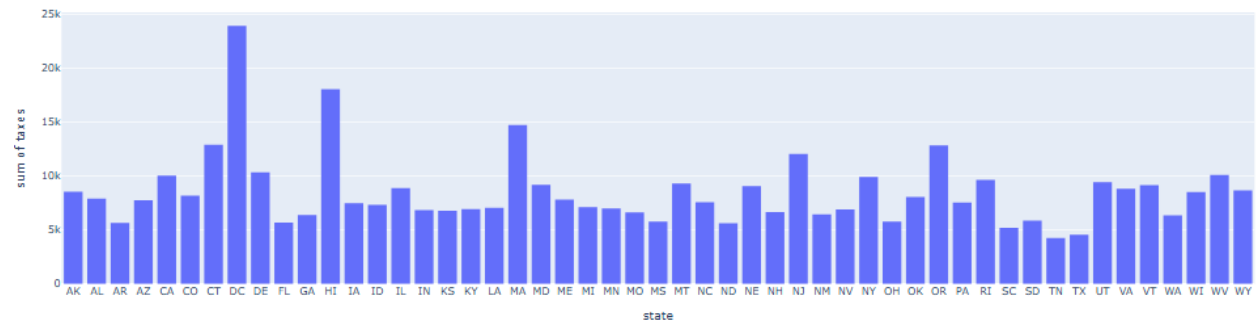


Figure 13

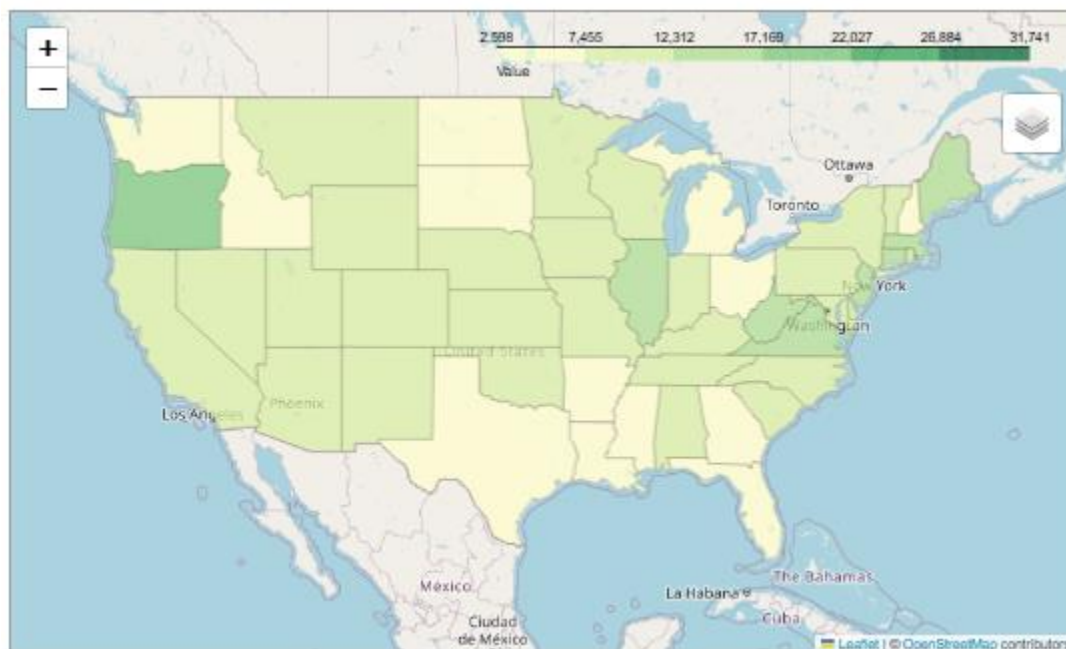


Figure 14



Income

Figure 15

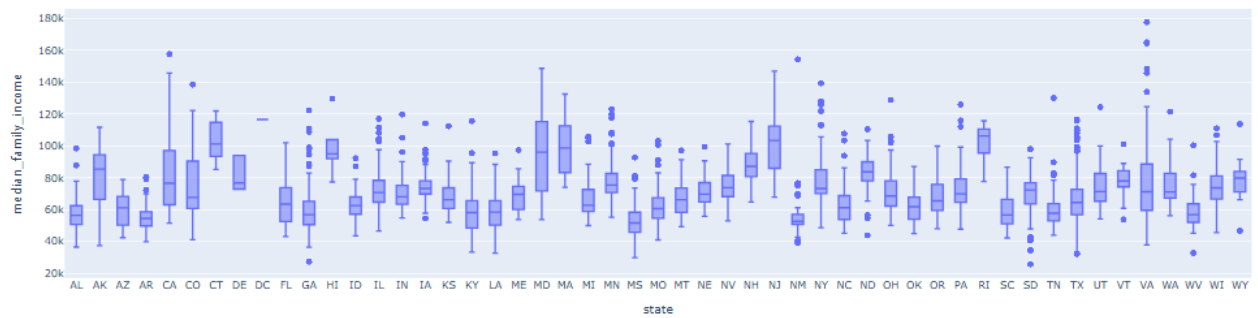


Figure 16

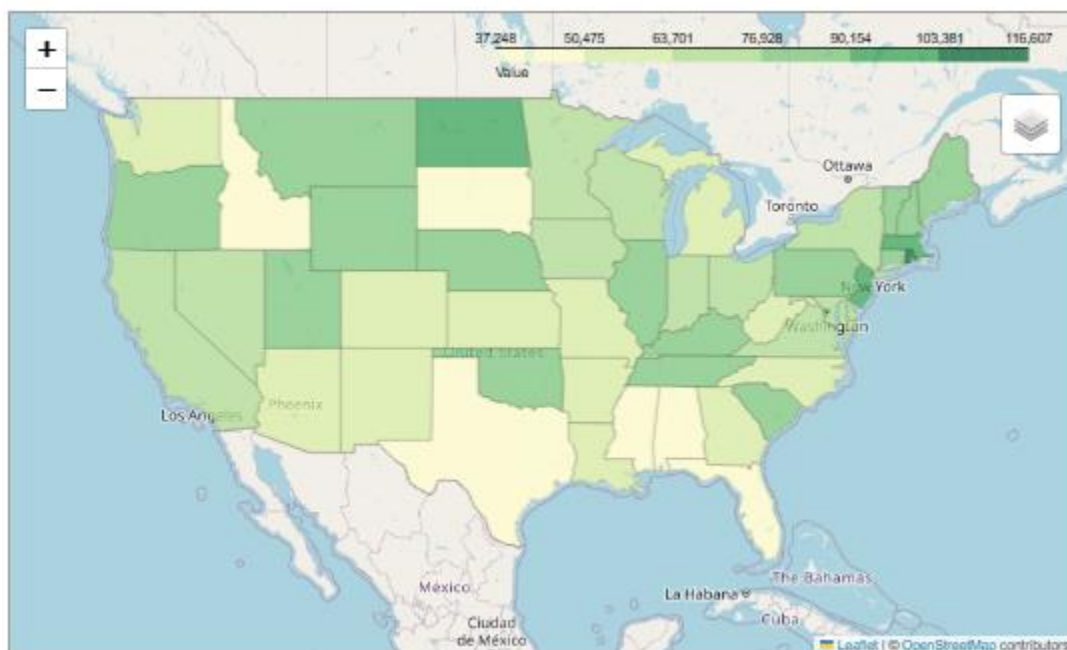
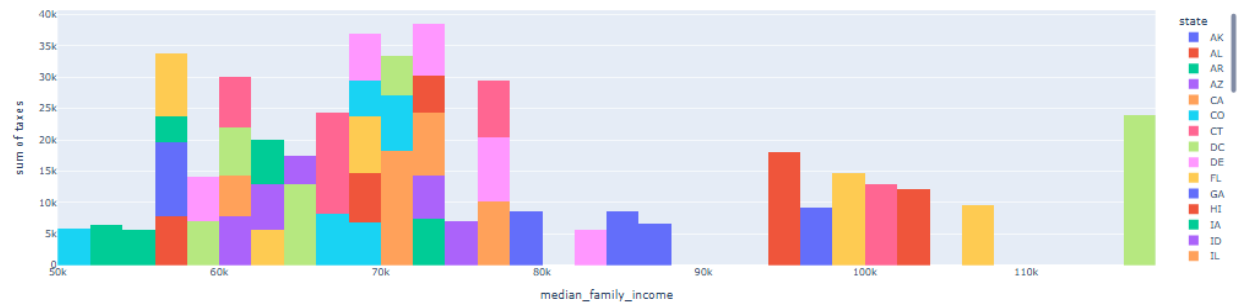


Figure 17



Summary

Figure 18



Figure 19

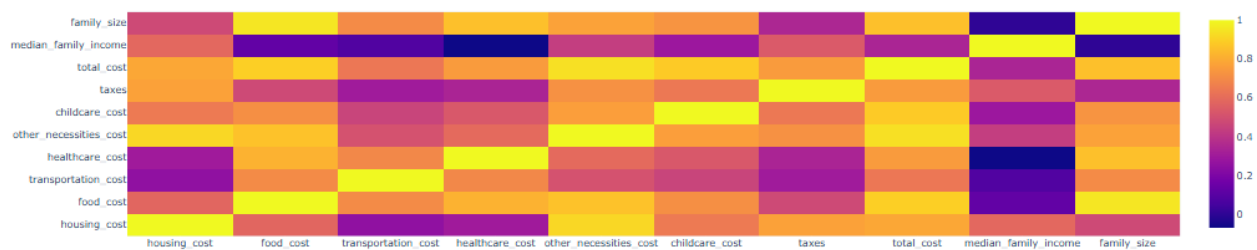


Figure 20

state	isMetro	areaname	county	family_m	housing_c	food_cost	transport	healthcar	other_nec	childcare	taxes	total_cost	median_f	state_full	parents	children	family_size	is_foodcos
AL	true	Montgome	Autauga	1p0c	8505.728	3454.917	10829.16	5737.479	4333.813	0	6392.945	39254.05	73010.41	Alabama	1	0	1	false
AL	true	Montgome	Autauga	1p1c	12067.50	5091.707	11588.19	8659.556	6217.458	6147.829	7422.078	57194.32	73010.41	Alabama	1	1	2	false
AL	true	Montgome	Autauga	1p2c	12067.50	7460.203	12361.77	11581.63	7075.658	15824.69	9769.562	76141.03	73010.41	Alabama	1	2	3	false
AL	true	Montgome	Autauga	1p3c	15257.15	9952.239	13452.18	14503.70	9134.356	18802.18	13101.70	94203.53	73010.41	Alabama	1	3	4	false
AL	true	Montgome	Autauga	1p4c	15257.15	12182.21	13744.59	17425.78	9942.363	18802.18	13469.21	100823.5	73010.41	Alabama	1	4	5	false

Appendix

Data_prep.py

```
import os
import copy
import json
import folium
import plotly.graph_objects as go
from urllib.request import urlopen

print (f"Current working directory: {os.getcwd()}")

STATE_FULLNAME_LOOKUP = {
    'AL': 'Alabama', 'AK': 'Alaska', 'AZ': 'Arizona', 'AR': 'Arkansas', 'CA':
    'California',
    'CO': 'Colorado', 'CT': 'Connecticut', 'DE': 'Delaware', 'DC': 'District of
    Columbia',
    'FL': 'Florida', 'GA': 'Georgia', 'HI': 'Hawaii', 'ID': 'Idaho', 'IL':
    'Illinois',
```

```

    'IN': 'Indiana', 'IA': 'Iowa', 'KS': 'Kansas', 'KY': 'Kentucky', 'LA':
    'Louisiana',
    'ME': 'Maine', 'MD': 'Maryland', 'MA': 'Massachusetts', 'MI': 'Michigan',
    'MN': 'Minnesota',
    'MS': 'Mississippi', 'MO': 'Missouri', 'MT': 'Montana', 'NE': 'Nebraska',
    'NV': 'Nevada',
    'NH': 'New Hampshire', 'NJ': 'New Jersey', 'NM': 'New Mexico', 'NY': 'New
    York',
    'NC': 'North Carolina', 'ND': 'North Dakota', 'OH': 'Ohio', 'OK': 'Oklahoma',
    'OR': 'Oregon',
    'PA': 'Pennsylvania', 'PR': 'Puerto Rico', 'RI': 'Rhode Island', 'SC': 'South
    Carolina',
    'SD': 'South Dakota', 'TN': 'Tennessee', 'TX': 'Texas', 'UT': 'Utah', 'VT':
    'Vermont',
    'VA': 'Virginia', 'WA': 'Washington', 'WV': 'West Virginia', 'WI':
    'Wisconsin', 'WY': 'Wyoming'
}

```

```
STATE_GEOMETRIES_URL =
```

```
"https://raw.githubusercontent.com/PublicaMundi/MappingAPI/master/data/geojson/us-states.json"
```

```
# import processing libraries
```

```
import numpy as np
```

```
import pandas as pd
```

```
import plotly.express as px
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.decomposition import PCA, TruncatedSVD, NMF
```

```
# Whether or not this project is being run on a local machine or in a Colab
notebook
```

```
LOCAL_ENV = True
```

```
DATASET_DIR = None
```

```
if(not LOCAL_ENV):
```

```
    from google.colab import drive
```

```
    drive.mount('/content/drive')
```

```
    DATASET_DIR = "/content/drive/MyDrive/datasets/"
```

```
else:
```

```
    DATASET_DIR = "data/"
```

```
# Helper functions
```

```

def apply_PCA(target: np.ndarray, num_components: int, standardize: bool = True)
-> np.ndarray:
    # auto-standardize if you wish
    if(standardize):
        target = StandardScaler().fit_transform(target)

    transformer = PCA(n_components=num_components)
    return transformer.fit_transform(target)

def apply_SVD(target: np.ndarray, num_components: int, standardize: bool = True)
-> np.ndarray:
    if(standardize):
        target = StandardScaler().fit_transform(target)

    U, S, VT = np.linalg.svd(target)

    # grab the left singular values
    U = U[:, :num_components]

    # grab the singular values
    S = np.diag(S[:num_components])

    #grab the right singular values
    VT = VT[:num_components, :]

    return np.dot(U, np.dot(S, VT))

def apply_NMF(target: np.ndarray, num_components: int, standardize: bool = True)
-> np.ndarray:
    if(standardize):
        target = StandardScaler().fit_transform(target)

    transformer = NMF(n_components=num_components)
    return transformer.fit_transform(target)

def load_dataset(dataset_name: str) -> pd.DataFrame:

    filename: str = f'{DATASET_DIR}{dataset_name}'
    print(f"Loading file {filename}...")
    df: pd.DataFrame = pd.read_csv(filename)

```

```

    return df

def clean_dataset(dataset: pd.DataFrame) -> pd.DataFrame:
    # drop nulls out-the-gate
    dataset = dataset.dropna()

    # append the full state names
    dataset['state_full'] = dataset['state'].map(STATE_FULLNAME_LOOKUP)

    # separate the household makeup strings into something meaningful using
    # regular expressions to parse the family_member_count string
    dataset["parents"] =
dataset["family_member_count"].str.extract(r"(\d+)p").astype(int)
    dataset["children"] =
dataset["family_member_count"].str.extract(r"p(\d+)c").astype(int)

    # and create a measure for family size, which is great for normalization for
    # more accurate county/state-wide averages. It won't be perfect
    # (eg, kids eat less than adults but this would assume equal rates) but it
    # will still work *well enough* for the purposes of this project.
    # I *could* count a child as half for something like food, or as nothing for
    # transportation, or half for housing, but that's all a guess
    # in any case.
    dataset["family_size"] = dataset["parents"] + dataset["children"]

    Q1 = dataset["food_cost"].quantile(0.25)
    Q3 = dataset["food_cost"].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    dataset["is_foodcost_outlier"] = (dataset["food_cost"] < lower) |
(dataset["food_cost"] > upper)

    total_null_entries = np.sum(dataset.isnull().sum())

    if(bool(total_null_entries != 0)):
        print(f"ERROR: no null values assertion failed. Total null entries:
{total_null_entries}")

```

```

        return None
    return dataset

def load_state_geometries():
    with urlopen(STATE_GEOMETRIES_URL) as response:
        state_geometries = json.load(response)
    return state_geometries

def create_choropleth_map(
    geo_data,
    dataframe,
    state_col_df, # Column in dataframe for state names
    value_col_df, # Column in dataframe for values to color code
    key_on_geojson, # GeoJSON property to join on (e.g.,
'feature.properties.name')
    map_location=[37.7749, -122.4194],
    map_zoom=4,
    fill_color_scheme='YlGn',
    legend_title='Value',
    map_path: str = "untitled"
):
    """Generates a Folium choropleth map.

    Args:
        geo_data (dict): The GeoJSON data for the map.
        dataframe (pd.DataFrame): DataFrame containing data to be mapped.
        state_col_df (str): Name of the column in `dataframe` that contains the
state identifiers.
        value_col_df (str): Name of the column in `dataframe` that contains the
values for color coding.
        key_on_geojson (str): The GeoJSON property to join on (e.g.,
'feature.properties.name').
        map_location (list): Latitude and longitude for the map's center.
        map_zoom (int): Initial zoom level of the map.
        fill_color_scheme (str): Colormap to use for the choropleth (e.g.,
'YlGn', 'RdBu', 'Plasma').
        legend_title (str): Title for the color legend.

    Returns:
        folium.Map: The generated Folium map object.
    """
    m = folium.Map(location=map_location, zoom_start=map_zoom)

```

```

# Make a copy of the GeoJSON data to avoid modifying the original
geo_data_copy = copy.deepcopy(geo_data)

# Extract the actual property key from key_on_geojson (e.g., 'name' from
'feature.properties.name')
geojson_prop_key = key_on_geojson.split('.')[ -1]

# Create a dictionary for quick lookup of values from the dataframe
data_lookup = dataframe.set_index(state_col_df)[value_col_df].to_dict()

# Iterate through GeoJSON features and add the value_col_df to their
properties
for feature in geo_data_copy['features']:
    state_name_in_geojson = feature['properties'].get(geojson_prop_key)
    if state_name_in_geojson in data_lookup:
        feature['properties'][value_col_df] =
data_lookup[state_name_in_geojson]
    else:
        # Handle cases where a state in GeoJSON might not be in the dataframe
        feature['properties'][value_col_df] = None # Assign None or a default
value

choropleth_layer = folium.Choropleth(
    geo_data=geo_data_copy, # Use the modified geo_data
    name='choropleth',
    data=dataframe, # Keep data and columns for choropleth coloring logic
    columns=[state_col_df, value_col_df],
    key_on=key_on_geojson,
    fill_color=fill_color_scheme,
    fill_opacity=0.7,
    line_opacity=0.2,
    legend_name=legend_title
).add_to(m)

# Now the GeoJsonTooltip can access value_col_df directly from
feature.properties
choropleth_layer.geojson.add_child(
    folium.features.GeoJsonTooltip(
        fields=[geojson_prop_key, value_col_df], # Now 'childcare_cost' is in
properties
        aliases=['State', legend_title], # Aliases for display
        localize=True,

```



```

        sticky=False,
        labels=True,
        style="""
            background-color: #F0EFEF;
            border: 2px solid black;
            border-radius: 3px;
            box-shadow: 3px;
        """,
        max_width=800,
    )
)

folium.LayerControl().add_to(m)
m.save(map_path)
return map_path

# def make_barchart_fig(df, x, y, title, agg="mean", color=None,
# colorscale=None):
#     return px.bar(data_frame=df, x=x, y=y, color = color, title = title)

def make_barchart_fig(df, x, y, title, agg="mean", color=None, colorscale=None):
    if agg:
        grouped = df.groupby(x, as_index=False)[y].agg(agg)
    else:
        grouped = df[[x, y]].copy()

    grouped = grouped.sort_values(by=y, ascending=False)

    # --- CASE 1: colorscale (data-driven) ---
    if colorscale:
        fig = px.bar(
            grouped,
            x=x,
            y=y,
            title=title,
            color=y,
            color_continuous_scale=colorscale
        )

```

```

        fig.update_coloraxes(
            cmin=grouped[y].min(),
            cmap=grouped[y].max() * 10
        )

# --- CASE 2: single color ---
else:
    fig = px.bar(
        grouped,
        x=x,
        y=y,
        title=title
    )

    if color:
        fig.update_traces(marker_color=color)

return fig

def make_histogram_fig(df, x, y, title, agg="mean", color=None, colorscale=None):
    if agg:
        grouped = df.groupby(x, as_index=False)[y].agg(agg)
    else:
        grouped = df[[x, y]].copy()

    grouped = grouped.sort_values(by=y, ascending=False)

# --- CASE 1: colorscale (data-driven) ---
fig = px.histogram(
    grouped,
    x=x,
    y=y,
    title=title,
    color=color
)
# fig.update_coloraxes(
#     cmin=grouped[y].min(),
#     cmap=grouped[y].max() * 10
# )
return fig

```

```

def make_interactive_table(df: pd.DataFrame, keys: list[str], labels: list[str],
num_entries: int) -> go.Figure:

    # --- validation ---
    if len(keys) != len(labels):
        raise ValueError("keys and labels must have the same length")

    missing = [k for k in keys if k not in df.columns]
    if missing:
        raise ValueError(f"Missing columns in dataframe: {missing}")

    # --- slice data ---
    df_slice = df.iloc[:num_entries]

    # --- build column values dynamically ---
    column_values = [df_slice[key] for key in keys]

    # --- create table ---
    fig = go.Figure(data=[go.Table(
        header=dict(values=labels),
        cells=dict(values=column_values)
    )])

    return fig


class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    x: float
    y: float


class Location(Point):
    label: str


def parse_family_size(s):
    return int(s[0]) + int(s[2])
    # import re
    # nums = list(map(int, re.findall(r'\d+', s)))
    # return sum(nums) if nums else None

```

```

def get_descriptives(df: pd.DataFrame) -> pd.DataFrame:
    # -----
    # type checking
    # -----
    if not isinstance(df, pd.DataFrame):
        raise TypeError(f"Expected pd.DataFrame, got {type(df)}")

    numeric_df = df.select_dtypes(include=[np.number]).copy()

    if numeric_df.empty:
        raise ValueError("No numeric columns found in DataFrame")

    results = []

    for col in numeric_df.columns:
        series = numeric_df[col].dropna()

        if series.empty:
            continue

        # basic stats
        col_min = series.min()
        col_max = series.max()
        col_range = col_max - col_min
        col_median = series.median()
        col_mean = series.mean()

        # mode (can be multiple; take first)
        mode_series = series.mode()
        col_mode = mode_series.iloc[0] if not mode_series.empty else np.nan

        # skew
        col_skew = series.skew()

        # IQR outliers
        q1 = series.quantile(0.25)
        q3 = series.quantile(0.75)
        iqr = q3 - q1
        lower = q1 - 1.5 * iqr
        upper = q3 + 1.5 * iqr

```

```
outlier_mask = (series < lower) | (series > upper)
outlier_count = outlier_mask.sum()
results.append({
    "column": col,
    "min": col_min,
    "max": col_max,
    "range": col_range,
    "mean": col_mean,
    "median": col_median,
    "mode": col_mode,
    "skew": col_skew,
    "outlier_count": outlier_count,
    "outlier_pct": outlier_count / len(series)
})

return pd.DataFrame(results)
```

Visualizations.py

```
import pandas as pd
import numpy as np
import dash
import seaborn as sns
from dash import dcc, html
from data_prep import *

from itertools import count

fig_counter = count(1)

def next_fig():
    return next(fig_counter)

def build_housing_section(state_df, df, geometries):
    items = []

    items.append(html.H1("Housing"))
    items.append(html.H2(f"Figure {next_fig()}"))
    items.append(dcc.Graph(
```

```

        figure=px.histogram(state_df, x="housing_cost", nbins=50)
    ))

    items.append(html.H2(f"Figure {next_fig()}"))
    items.append(html.Iframe(
        srcDoc=open(create_choropleth_map(
            geometries, df, 'state_full', 'housing_cost',
            'feature.properties.name', [37.7749, -95.7129], 4
        )).read(),
        width="800", height="480"
    ))

    items.append(html.H2(f"Figure {next_fig()}"))
    items.append(dcc.Graph(
        figure=px.scatter(state_df, x="housing_cost", y="taxes", color="state")
    ))

    return items

def build_transportation_section(state_df, df, geometries):
    items = []

    items.append(html.H1("Transportation"))

    items.append(html.H2(f"*Figure {next_fig()}"))
    items.append(dcc.Graph(
        figure=px.histogram(state_df, x="transportation_cost", nbins=50)
    ))

    items.append(html.H2(f"*Figure {next_fig()}"))
    items.append(html.Iframe(
        srcDoc=open(create_choropleth_map(
            geometries, df, 'state_full', 'transportation_cost',
            'feature.properties.name', [37.7749, -95.7129], 4
        )).read(),
        width="800", height="480"
    ))

    items.append(html.H2(f"Figure {next_fig()}"))
    items.append(dcc.Graph(
        figure=px.scatter(
            state_df,
            x="transportation_cost",

```

```

        y="total_cost",
        trendline="ols"
    )
))

return items

def build_healthcare_section(state_df, df, geometries):
    items = []

    items.append(html.H1("Healthcare"))

    items.append(html.H2(f"Figure {next_fig()}"))
    items.append(dcc.Graph(
        figure=px.histogram(state_df, x="healthcare_cost", nbins=50)
    ))

    items.append(html.H2(f"Figure {next_fig()}"))
    items.append(html.Iframe(
        srcDoc=open(create_choropleth_map(
            geometries, df, 'state_full', 'healthcare_cost',
            'feature.properties.name', [37.7749, -95.7129], 4
        )).read(),
        width="800", height="480"
    ))

    return items

def build_income_section(state_df, df, geometries):
    items = []

    items.append(html.H1("Income"))
    items.append(html.H2(f"Figure {next_fig()}"))
    items.append(dcc.Graph(
        figure=px.box(df, x="state", y="median_family_income")
    ))

    items.append(html.H2(f"Figure {next_fig()}"))
    items.append(html.Iframe(
        srcDoc=open(create_choropleth_map(
            geometries, df, 'state_full', 'median_family_income',
            'feature.properties.name', [37.7749, -95.7129], 4
        )).read(),
        width="800", height="480"
    ))

```



```

))

items.append(html.H2(f"Figure {next_fig()}"))
items.append(
    dcc.Graph(

        figure = px.histogram(
            state_df,
            x="median_family_income",
            y="taxes",
            color = "state",
            nbins=60
        )
    )
)

return items

def build_childcare_section(state_df, df, geometries, childcare_sorted_df):
    items = []

    items.append(html.H1("Childcare"))
    items.append(html.H2(f"Figure {next_fig()}"))
    items.append(html.Iframe(
        srcDoc=open(create_choropleth_map(
            geometries, df, 'state_full', 'childcare_cost',
            'feature.properties.name', [37.7749, -95.7129], 4
        )).read(),
        width="800", height="480"
    ))
    items.append(html.H2(f"Figure {next_fig()}"))
    items.append(dcc.Graph(
        figure=make_barchart_fig(
            childcare_sorted_df,
            "county",
            "childcare_cost",
            "Top 12 Highest Childcare Cost Counties"
        )
    ))
    items.append(html.H2(f"Figure {next_fig()}"))
    items.append(
        dcc.Graph(

```

```

        figure = px.histogram(
            state_df,
            x="childcare_cost",
            y="taxes",
            color = "state",
            nbins=35
        )
    )
)

return items

def build_tax_section(state_df, df, geometries, tax_sorted_df):
    items = []

    items.append(html.H1("Taxes"))
    items.append(html.H2(f"Figure {next_fig()}"))
    items.append(dcc.Graph(
        figure=px.histogram(state_df, "state", "taxes")
    ))
    items.append(html.H2(f"Figure {next_fig()}"))
    items.append(html.Iframe(
        srcDoc=open(create_choropleth_map(
            geometries, df, 'state_full', 'taxes',
            'feature.properties.name', [37.7749, -95.7129], 4
        )),read(),
        width="800", height="480"
    ))
    items.append(html.H2(f"Figure {next_fig()}"))
    items.append(dcc.Graph(
        figure=make_barchart_fig(
            tax_sorted_df,
            "county",
            "taxes",
            "Top 12 Highest-Tax Counties"
        )
    ))

    return items

def build_summary_section(df, state_df, corr):
    items = []

```

```

#items.append(dcc.Graph(go.Figure(data=[go.Table(
#header=dict(values=["areaname", "county", "median_family_income"],
#               fill_color='paleturquoise',
#               align='left'),
#cells=dict(values=[df.areaname, df.county, df.median_family_income],
#              fill_color='lavender',
#              align='left'))
#])))

items.append(html.H1("Summary"))

# 10.
items.append(html.H2(f"Figure {next_fig()}"))
items.append(dcc.Graph(
    figure=px.scatter(state_df, x="total_cost", y="taxes", trendline="ols")
))

items.append(html.H2(f"Figure {next_fig()}"))
items.append(dcc.Graph(
    figure=go.Figure(
        data=go.Heatmap(
            z=corr.values,
            x=corr.columns,
            y=corr.columns
        )
    )
))

items.append(html.H2(f"Figure {next_fig()}"))
items.append(
    dcc.Graph(
        figure=go.Figure(
            data=[go.Table(
                header=dict(
                    values=list(df.columns),
                    fill_color='paleturquoise',
                    align='left'
                ),
            ),
            cells=dict(
                values=[df[col] for col in df.columns],
                fill_color='lavender',

```

```

        align='left'
    )
    )]
),
config={"displayModeBar": False}
)
)

return items

def build_title():
    items = []
    items.append(html.H1("Report on The Cost of Living (US)"))
    items.append(html.H2("By Devin Frost"))
    return items

def build_analysis():
    items = []

    intro = """
        In recent years, the US economy has been hit hard by multiple economic
        crises, from global pandemics to supply chain shocks to wars, and naturally, the
        expenditure of resources to combat these crises has negatively impacted the cost
        of living in the US. This report takes a holistic look into the cost of living in
        the United States, using data from the
        Economic Policy Institute (EPI) In order to understand the state of the US
        economy, we first need to measure the cost of living, then normalize it in
        accordance to median incomes in order to gauge regional affordability.
        """

    items.append(html.P(intro))

    data_prep = """
        To prepare my data for this analysis, I dropped rows with null values to
        simplify the analysis process. It only meant losing 10 records, which should not
        meaningfully offset the value of this analysis.

        I also synthesized numerous measures using the dataset. The dataset gives a
        simple 'family_member_count' feild, which isn't provided as a numerical value. I
        parse the values into the `parents` and `children` feilds.

        Further, I sum these up into a `family_size` value. In order to measure the
        affordability of a region, I take total expenditures and divide it by median
        income, to normalize the values.
    """

```

```

"""
items.append(html.P(data_prep))

childcare = """
The family unit is not complete without children, and in the modern, fast-
paced world, childcare is a necessity. Figure 1 is a map of
childcare costs by state. Childcare costs have considerable variation across
the states. Figure 2 is a bar chart of the top 12 most
expensive counties in which to raise a child. It includes counties from New
York and the District of Columbia.
"""
items.append(html.P(childcare))

# Analysis
housing = """
Housing. We all need it, whether we are high up in the mountains of
Appalachia or in the Great Plains - to simply survive the elements.
Figure 4 is a histogram that shows the typical annual cost of housing per
state. Figure 5 shows housing prices by state on a map,
and as you can see, housing prices are generally highest near the coasts, and
lowest in the Midwest and Southeast. Hawaii, being a small
slice of Heaven on Earth, naturally has the highest housing prices, with a
median of 61k / year. Country-wide it floats at a median of
$10,400 each year. Figure 6 is a scatterplot of housing prices versus taxes,
and suggests that higher housing costs is positively correlated
with higher taxes, which is likely due to property tax law.
"""
items.append(html.P(housing))

transportation = """
A well-functioning economy depends on cheap transportation to move goods,
services, and people to where the demand is.
Figure 7 shows the distribution curve of transportation costs across America.
As you can see, the graph is left skewed
with a median of approximately $13,800 a year, which, as shown by Figure 8 is
due the the outlier Washington, D.C.,
where transportation is cheap. Figure 9 is a regression plot of
transportation cost versus total expenditures, and it
shows that transportation costs are negatively correlated with total costs.
This may indicate that cheaper transportation

```

```

    leads to more economic growth and therefore higher paying jobs and cost of
    living
    """
    items.append(html.P(transportation))

    healthcare = """
    Our quality of life is impacted largely by our health, and consequently,
    regions with high healthcare costs are
    indicative of regions that struggle to meet a standard of living. Figure 10
    is a histogram of the distribution of healthcare
    expenditures accross the country. It is somewhat right-skewed, but is largely
    normally distributed. Figure 11 shows that West
    Virginia is by far the most expensive place to pay for healthcare, with a
    median price of $37,000 annually. It is reflective of
    poor policy decisions in the region.
    """
    items.append(html.P(healthcare))


    taxes = """
    Unfortunately, in order to organize society, any state needs a government,
    and any government needs tax dollars to power programs that
    power public infrastructure and social programs. Figure 12 is a bar chart of
    taxes by state, and Figure 13 is a map thereof. As shown
    by the map, Oregon and Washington, DC have some of the highest taxes in the
    country. Figure 14 is a bar chart of the top 12 highest-tax
    counties in the country. Counties from New York and Washington, D.C have the
    highest taxes of any region in the United States
    """
    items.append(html.P(taxes))

```

```

income = """
    In order to pay for all of this, people naturally need sturdy, competitive
    incomes. Figure 15 is a boxplot of median household
    incomes across the counties of the 50 United States, plus the District of
    Columbia, and it shows that incomes across the state are
    highly variable, ranging from $27,000 in Georgia, up to $177k in Virginia.
    Virginia was by far the most riddled with inter-county
    income inequality. Delaware had the least variation. Figure 16 better conveys
    this information geographically. Figure 17 is a histogram
    of median family incomes by state and it is evident that the median wage in
    the US is approximately $70,000 a year.

    """

items.append(html.P(income))
return items

def build_conclusion():
    items = []
    income = """
        Figure 19 is a correlation matrix between each of the aforementioned factors
        and concludes that family size is highly
        correlated with the cost of food and healthcare, that total costs are highly
        correlated with the cost of food and the cost of
        extraneous necessities, and, interestingly, housing costs are correlated with
        extraneous necessity expenditures. Figure 18 shows
        that total expenditures are positively correlated with higher taxes. A small
        table is provided to illustrate the data that were
        provided by the Economic Policy Institute (EPI), including the aforementioned
        feilds `parents`, `children`, `family_size`, and
        `is_foodcost_outlier`.
        """

    items.append(html.P(income))
    return items

```

```

def make_dashboard(df, state_df, name = "Cost Of Living Dashboard") -> dash.Dash:

    items = []
    state_df["affordability"] = state_df["total_cost"] /
state_df["median_family_income"]

    geometries = load_state_geometries()

    app = dash.Dash(name)

    stacked_df = (
df.groupby(["state", "family_member_count"])
.size()
.reset_index(name="count")
)

    # Normalize to percentages within each state
    stacked_df["percent"] = (
        stacked_df["count"] /
        stacked_df.groupby("state")["count"].transform("sum")
    )

    summary_df = (
        stacked_df.groupby("family_member_count", as_index=False)["percent"]
        .mean()
    )

    state_order = (
        stacked_df.groupby("state")["count"]
        .sum()
        .sort_values(ascending=False)
        .index
    )

    family_df = (
        df.groupby("family_member_count", as_index=False)
        .mean(numeric_only=True)
    )

```



```

childcare_sorted_df = (
    df.groupby("county", as_index=False)["childcare_cost"]
        .mean()
        .sort_values("childcare_cost", ascending=False)
        .head(12)
)

tax_sorted_df = (
    df.groupby("county", as_index=False)["taxes"]
        .mean()
        .sort_values("taxes", ascending=False)
        .head(12)
)

corr_df = pd.DataFrame()

corr_df = df.copy()

corr_df.drop(columns=["case_id", "state", "isMetro", "areaname", "county",
"family_member_count", "state_full", "parents", "children", "is_foodcost_outlier"],
inplace=True, errors="ignore")

corr = corr_df.corr()

# Title
items += build_title()
items += build_analysis()
items += build_conclusion()

# Sections
items += build_childcare_section(state_df, df, geometries,
childcare_sorted_df)
items += build_housing_section(state_df, df, geometries)
items += build_transportation_section(state_df, df, geometries)
items += build_healthcare_section(state_df, df, geometries)
items += build_tax_section(state_df, df, geometries, tax_sorted_df)
items += build_income_section(state_df, df, geometries)

```

```

items += build_summary_section(df, state_df, corr)

app.layout = html.Div(items)
return app

# for my reference
'''
        column      min      max      range      mean
median      mode      skew  outlier_count  outlier_pct
0           case_id  1.000000  3171.00000  3170.000000  1589.329726
1593.500000  1.00000  -0.005906      0  0.000000
1           housing_cost  4209.311280  61735.58760  57526.276320
11073.359622  10416.000000  8808.00000  2.575029      1765  0.056174
2           food_cost  2220.276840  31178.61960  28958.342760
8287.654372  8129.156280  3112.53768  0.317386      66  0.002101
3  transportation_cost  2216.461440  19816.48200  17600.020560
13593.989839  13698.315000  14366.26920  -0.471037      227  0.007225
4           healthcare_cost  3476.379960  37252.27440  33775.894440
13393.008581  13082.514000  5506.79004  0.446041      316  0.010057
5  other_necessities_cost  2611.642080  28829.44320  26217.801120
7015.258917  6733.056120  4925.60592  1.166022      567  0.018046
6           childcare_cost  0.000000  48831.08520  48831.085200  9879.247416
10166.340120  0.00000  0.302431      272  0.008657
7           taxes  1027.800756  47753.39040  46725.589644
7657.101541  6897.747780  6914.53932  2.603167      1506  0.047931
8           total_cost  30087.662400  223717.54800  193629.885600
70899.620274  70974.249000  35171.52840  0.483501      240  0.007638
9  median_family_income  25529.976562  177662.46875  152132.492188
68315.997017  65955.605469  62893.09375  1.268878      1180  0.037556
10          parents  1.000000  2.00000  1.000000  1.500000
1.500000  1.00000  0.000000      0  0.000000
11          children  0.000000  4.00000  4.000000  2.000000
2.000000  0.00000  0.000000      0  0.000000
12          family_size  1.000000  6.00000  5.000000  3.500000
3.500000  2.00000  0.000000      0  0.000000

'''

```

main.py

```
from visualizations import make_dashboard
from data_prep import *

if __name__ == "__main__":
    # first things first, clean the dataset
    df = clean_dataset(load_dataset("cost_of_living_us.csv"))

    stacked_df = (
        df.groupby(["state", "family_member_count"])
          .size()
          .reset_index(name="count")

    )

    state_df = (
        df.groupby(["state", "state_full"], as_index=False)
          .agg({
              "housing_cost": "mean",
              "food_cost": "mean",
              "transportation_cost": "mean",
              "healthcare_cost": "mean",
              "other_necessities_cost": "mean",
              "childcare_cost": "mean",
              "taxes": "mean",
              "total_cost": "mean",
              "median_family_income": "mean",
```

```

    })
)

# secondary attempt
numeric_cols = df.select_dtypes(include=np.number).columns
state_df = (
df.groupby(["state", "state_full"], as_index=False)[numeric_cols]
    .median()
)

Q1 = state_df["food_cost"].quantile(0.25)
Q3 = state_df["food_cost"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

state_df["is_foodcost_outlier"] = (state_df["food_cost"] < lower) |
(state_df["food_cost"] > upper)

stacked_df = stacked_df.sort_values(["state", "family_member_count"])
state_order = (
    stacked_df.groupby("state")["count"]
        .sum()
        .sort_values(ascending=False)
        .index
)

dashboard = make_dashboard(df, state_df, "Cost Of Living Dashboard")

dashboard.run(debug=True)

```